

hermetic_webdav_run.sh — companion guide (§11.4.18)

Revision: 1 **Last modified:** 2026-07-02T00:00:00Z **Status:** H2.webdav for the hermetic WireGuard test harness ([design](#)) — the third §11.4.52 **promotion** of an operator-gated protocol test to autonomous (after the [Chromecast eureka leg](#) and the [FTP leg](#)). Authority: inherits constitution/Constitution.md per §11.4.35.

Overview

tests/vpn_lan/hermetic_webdav_run.sh promotes the **WebDAV leg** of the operator-gated tests/vpn_lan/ftp_sftp_webdav.sh to run **autonomously** (§11.4.52). That leg does `curl -x "$HELIX_SQUID_PROXY" -X PROPFIND "$HELIX_VPN_WEBDAV_URL"` and requires **HTTP 207 Multi-Status** + a non-empty XML body (a real non-207 response is a **FAIL**, fail-closed §11.4.68). Because it routes **through a proxy**, the hermetic promotion stands up TWO pure-python-stdlib servers inside one unshare -Urm:

1. a **WebDAV origin** in netns B bound to the WG-only overlay 10.10.0.2:8080 (`http.server.BaseHTTPRequestHandler` with `do_PROPFIND` → 207 + a valid `<D:multistatus>` body embedding this run's nonce; RFC 4918);
2. a **forward proxy** in netns A on 127.0.0.1:3128 (a raw-socket local-Squid stand-in) that accepts curl's absolute-form request line (`PROPFIND http://10.10.0.2:8080/dav/ HTTP/1.1`, RFC 7230 §5.3.2), re-emits origin-form upstream (§5.3.1), and streams the response back.

Then it runs the **UNMODIFIED** `ftp_sftp_webdav.sh` inside netns A: its `bridge_require` gate flips SKIP→UP and its WebDAV leg produces a **real 207 PASS** over the encrypted tunnel through the proxy — no operator, no podman, no Squid, no Mullvad. **Zero host installs** (§11.4.122): both servers are stdlib (`http.server` + `socketserver`).

How the bridge contract is pointed at the hermetic peer

var	hermetic value
HELIX_SWORD_DIR	the served dir (non-empty)
HELIX_BRIDGE_CONNECT / _DISCONNECT	true
HELIX_BRIDGE_HEALTH	a real TCP probe of 10.10.0.2:8080 over the tunnel
HELIX_BRIDGE_SUBNET	10.10.0.0/24
HELIX_BRIDGE_HOST	10.10.0.2
HELIX_VPN_WEBDAV_URL	http://10.10.0.2:8080/dav/
HELIX_SQUID_PROXY	http://127.0.0.1:3128 (the hermetic forward proxy — no credentials, §11.4.10)

The origin binds **ONLY** the overlay 10.10.0.2 (never the underlay 10.9.0.2 / 0.0.0.0), so reaching it from netns A is possible only through wg0 — a successful PROPFIND is itself proof of tunnel traversal (§11.4.111). HELIX_BRIDGE_MODE=hermetic documents intent; the promotion is realised through the contract vars. The FTP + SFTP legs of the same test SKIP honestly (their vars unset).

Usage

```
tests/vpn_lan/hermetic_webdav_run.sh # PASS /  
SKIP / FAIL  
WEBDAV_MUT=bad207 tests/vpn_lan/hermetic_webdav_run.sh # §1.1  
golden-bad
```

Evidence: qa-results/vpn_lan/hermetic_webdav/<UTC-ts>_<pass|mut>_<pid>/run.evidence (+ selffetch.propfind.xml; the promoted test also writes qa-results/vpn_lan/phase3/.../webdav/).

Anti-bluff design

The normal PASS is emitted **only** when the promoted ftp_sftp_webdav.sh exits 0 **and** its stdout carries a real ^PASS: .*WebDAV line — a SKIP-only run can never satisfy that. The **golden-bad** (WEBDAV_MUT=bad207) makes the origin answer PROPFIND with HTTP **200** instead of 207; the only acceptable outcome is the real test's fail-closed branch firing (^FAIL: .*WebDAV, exit ≠ 0). This proves the 207 assertion is genuinely exercised, not a rubber-stamp (§11.4.107(10) / §11.4.68).

Self-fetch cross-check (§11.4.107 not-stale). Before running the promoted test, the harness PROPFINDs `http://10.10.0.2:8080/dav/` through the proxy **itself** and asserts 207 + THIS run's fresh nonce in the `<D:multistatus>` body (normal) / a non-207 (golden-bad). This ties the eventual PASS to a real 207 round-trip of *our* data over the encrypted overlay **through the proxy**, decoupled from the downstream grep string and forbidding a stale/wrong origin. A **coupling-contract guard** (`grep -q 'WebDAV' "$TEST"`) makes a future rename of the promoted leg fail with a clear diagnostic here.

Host-safety self-bounding. Both servers run under `timeout -k 2 90` (the origin inside netns B, the proxy in netns A) so an outer-SIGKILL orphan self-terminates (§12); the outer `timeout 80` reaps the whole unshare tree first. The proxy forces `Connection: close` upstream so the relay loop terminates cleanly (no keep-alive hang).

Ambient-env determinism (§11.4.50). Before invoking the promoted test the harness unsets the FTP + SFTP leg contract vars, so a leftover ambient-shell export cannot make a sibling leg attempt a (from-netns unreachable) real connection and flip the promoted test's exit code into a spurious FAIL. This is a false-negative guard only — it can never manufacture a bluff PASS — and pins the promoted-test environment to exactly the WebDAV leg.

Captured evidence (verified 2026-07-02)

Normal run embedded the promoted test's own output:

```
self-fetch PROPFIND through proxy: http_status=207 body_bytes=779
ftp_sftp_webdav: sword bridge UP — running live FTP/SFTP/WebDAV
checks (subnet=10.10.0.0/24 host=10.10.0.2)
PASS: WebDAV PROPFIND via existing Squid (expect 207 Multi-Status)
[evidence: .../phase3/.../webdav/propfind.evidence]
HW_PASS: the UNMODIFIED ftp_sftp_webdav.sh WebDAV leg produced a
REAL 207 PASS over the hermetic WireGuard tunnel through a pure-
stdlib forward proxy
```

3/3 deterministic (§11.4.50). Golden-bad → the real test FAILED
fail-closed: FAIL: WebDAV PROPFIND ... returned HTTP 200 (expected 207).

Honest scope (§11.4.6 / §11.4.3)

Proves the **WebDAV PROPFIND 207 logic** (through a proxy hop) autonomously over an encrypted tunnel against a controlled origin. It does NOT prove the real Squid on the real Mullvad topology (that stays the §11.4.3 operator-gated confirmation), and the 207 body is a controlled fixture (it exercises the client-side PROPFIND/status assertion, not a full WebDAV server implementation).

Prerequisites / SKIP

bash, unshare+nsenter, iproute2 ip (wireguard link type), wg, host wireguard kernel module, python3 (stdlib only), curl, timeout, plus tests/vpn_lan/ftp_sftp_webdav.sh and tests/lib/sword_bridge.sh. Any missing → honest SKIP: (§11.4.3). Process-headroom guard SKIPS on a starved host (§12).

Security (§11.4.10)

WireGuard private keys: mode-0600 mktemp inside the namespace, used by path, removed on exit, never logged. WebDAV is unauthenticated — no credentials anywhere. The <D:multistatus> body carries a per-run nonce (fresh value, §11.4.107 not-stale).

Provenance (§11.4.147)

Drafted by a builder subagent that crashed on a session-limit before self-testing; the conductor resumed the residue-clean partial per §11.4.147 (work not lost), verified it empirically GREEN (normal + golden-bad + 3/3 determinism), and passed it through the independent §11.4.142 review before commit.

Underlay-sniff non-leak differential (§11.4.107 / FINDINGS §7.1)

During the round-trip the harness captures on the underlay veth0 (rootless AF_PACKET; tcpdump fallback; honest §11.4.3 SKIP if neither) and asserts BOTH (a) WG ciphertext present — a type-4 0x04 datagram to the WG listen port :51820 — AND (b) the per-run WebDAV marker (\$NONCE, in the 207 body) is ABSENT in the raw underlay bytes. Only the proxy→origin 10.10.0.2:8080 hop rides wg0/veth0 (client→proxy is loopback, irrelevant). Verbatim single-source clone of the substrate analyzer (_emit_an_py, ethertype-guarded). The load-bearing golden-bad **SNIFF_MUT=plain** emits

\$NONCE as cleartext UDP to the discard port 10.9.0.2:9 (distinct from the §11.4.111 HTTP negative-control port :8080, so NEG-OK stays valid) → ONLY the plaintext-absent assertion flips to FAIL while ciphertext stays present, proving the sniff is not a tautology (§11.4.107(10)). Landed 85d8b32, independent review aldca6fd GO.

Related

- [hermetic_ftp_run.md](#) — the sibling H2.x FTP promotion.
- [hermetic_bridge_run.md](#) — the H2 template (Chromecast eureka leg).
- [hermetic_wg_roundtrip.md](#) — the H0-full tunnel this reuses.
- [tests/vpn_lan/ftp_sftp_webdav.sh](#) — the promoted protocol test.
- [../research/hermetic_protocol_promotion_20260702/FINDINGS.md](#) — the RFC recipe this follows.

Last verified

2026-07-02 (host: ALT Linux kernel 6.12, wireguard module, wireguard-tools 1.0.20210914, python3 stdlib, curl).